

Multi-Party Authorization Proxy

Georgia Tech — CS 6727 Cybersecurity Practicum

Ian Barish

AI Use: Project, deck, and demo created with help from Claude & GPT 5.6; research, design, and all project decisions are my own.

Multi-party authorization is an **underused** security control

WHAT IS IT?

Requiring more than one person to approve before an action goes through.

WHERE YOU ALREADY SEE IT



Crypto multisig

moving funds



AWS

backup vaults · key imports



Google Cloud

privileged IAM grants



Azure

backups · device configs

My project: a general-use multi-party authorization proxy

My headline use case: package publishing

Case Study

September 14, 2025 - An npm supply-chain worm begins.

1

stolen publish credential — all it needed to start

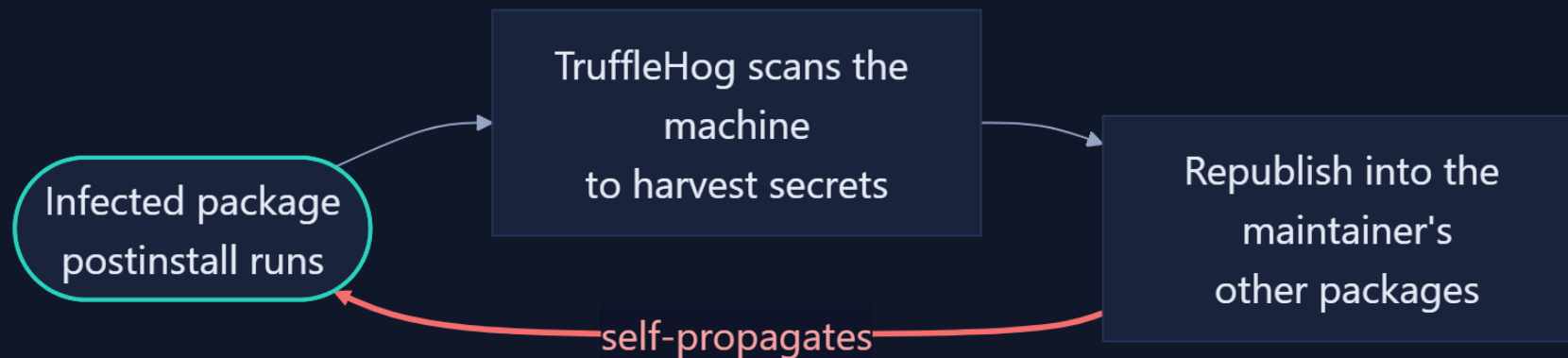
~500

packages compromised in roughly a day

2M+

weekly downloads on a single infected package

HOW IT SPREADS — AFTER THE TOKEN IS STOLEN



They named it Shai-Hulud

The worm dumps every stolen secret into a public GitHub repo it names *Shai-Hulud* — the sandworm from *Dune*.

Sep 2025

Patient Zero · ~500 packages

Nov 2025

"Second Coming" · ~700 packages · 25k+ repos

Spring 2026

TanStack · valid SLSA L3

Jun 2026

Red Hat packages

Jul 2026

AsyncAPI · bypassed all review



Dune: Part Two © Warner Bros. Pictures

Multi-Party Authorization Proxy · CS 6727 Final Presentation

WHAT NPM & GITHUB BUILT IN RESPONSE

- Mandatory 2FA on local publishing
- Granular tokens, 7-day lifespans
- Trusted Publishing, no long-lived tokens
- FIDO hardware keys replacing TOTP

Every fix hardened **who publishes** — and it kept surfacing.

Not just npm. Not just one worm

Every control the industry built to stop a poisoned release, against every way a release gets poisoned.

Control and the decision it actually gates	Stolen credential Shai-Hulud	Trusted insider XZ backdoor	Compromised CI poisoned pipeline	Direct publish skip repo + CI
Mandatory 2FA / MFA proves login identity	~	X	X	X
Trusted Publishing (OIDC) proves publish identity	✓	X	X	~
Build provenance (SLSA) attests build integrity	X	X	X	X
Required reviews + branch protection gates the repo merge	✓	~	X	X
CI/CD deployment gates gates the pipeline deploy	✓	~	~	X
Artifact-repo promotion gates internal promotion	✓	~	~	X

✓ covers it · ~ partial · X misses it

The missing layer: authorization

Control and the decision it actually gates	Stolen credential Shai-Hulud	Trusted insider XZ backdoor	Compromised CI poisoned pipeline	Direct publish skip repo + CI
Mandatory 2FA / MFA proves login identity	~	X	X	X
Trusted Publishing (OIDC) proves publish identity	✓	X	X	~
Build provenance (SLSA) attests build integrity	X	X	X	X
Required reviews + branch protection gates the repo merge	✓	~	X	X
CI/CD deployment gates gates the pipeline deploy	✓	~	~	X
Artifact-repo promotion gates internal promotion	✓	~	~	X
The proxy authorizes the release itself	✓	✓	✓	✓*

✓ covers it · ~ partial · X misses it · ✓* under one deployment precondition, revisited later

How the proxy authorizes a release

It sits in front of the registry: an upload is *held, not published*, until the package's owners authorize the exact artifact.

1

Request

Published via **twine** — to the proxy, not the registry.

2

Hash-bind

Artifact **hashed** and held; the approval is **bound** to it.

3

Approve

Each owner **re-auths** and **signs** their vote.

4

Quorum

Waits for **m-of-n**. One denial stops it.

5

Publish

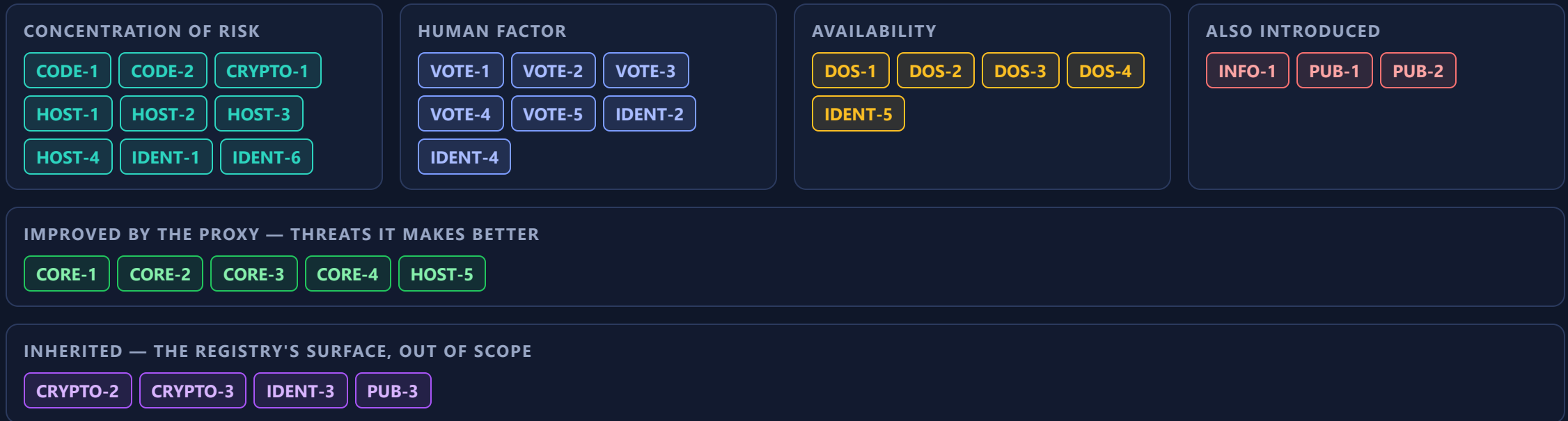
Hash **re-checked**, then released to PyPI.

The owners approve **one exact artifact** — and that's the artifact that ships.

Demo

Aren't we adding new risks?

Yes. One gate in front of every release concentrates risk — the biggest new threat the project has. So here's the *whole* surface it introduces: all 33 threats, catalogued.



- Concentration — the gate is one target
- Human factor — fatigue · coercion · replay
- Availability — the gate as a jam point
- Other — outside the three themes
- Improved — threats the proxy makes better
- Inherited — out of scope, the registry's

It only works if it's deployed right

The proxy's guarantees rest on a few steps it *can't enforce for you*. Two carry the most weight.

Revoke every pre-existing publish token. The whole guarantee rests on the proxy holding the *only* credential that can upload — leave an old one live and an attacker skips the gate entirely.

PUB-2 · proxy bypass

Choose the quorum and the co-owners deliberately. High enough to mean something, low enough that losing one approver can't freeze a release — and choose co-owners that are unlikely to collude.

DOS-3 · DOS-4 · CORE-3

... ~two dozen more — the full operator checklist ships with the proxy

What I'm not claiming

A fully colluding quorum

If every required co-owner conspires, the proxy approves the release — it did exactly what they told it to. Raising the bar from one person to several was never a promise to survive everyone you trusted turning at once.

CORE-3 · out of scope by design

The registry's own surface

The proxy sits *in front of* a registry it doesn't own, so it inherits that registry's weaknesses. Take over the PyPI account through its recovery flow and you never touched the proxy at all.

PUB-3 · inherited

Future Work

Where this use case goes next

Build it into the registry. As a native feature, the biggest risk the proxy introduced dissolves — the gate *becomes* the registry, already the juiciest target in the ecosystem. Trusted Publishing is proof the community will change the publish path when the case is strong.

Multi-party authorization as a whole

An option in more places. It belongs anywhere one stolen credential can trigger something you can't take back: a production deploy, a root cloud account, a wire transfer. Today almost nothing offers it.

For anything you can't take back, **one stolen credential shouldn't be enough** — and now it doesn't have to be.



LET'S CONNECT